



Día, Fecha:	JUEVES, 26/03/2026
Hora de inicio:	17:20

ANÁLISIS Y DISEÑO DE SISTEMAS 2 SECCIÓN B

Escuela de Ingeniería en Ciencias y Sistemas

NOMBRE DEL TUTOR



Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería de Ciencias y Sistemas

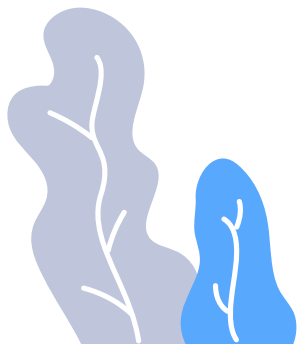


Pruebas de software

Análisis y Diseño de Sistemas 2

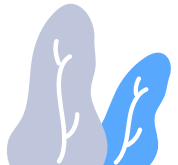
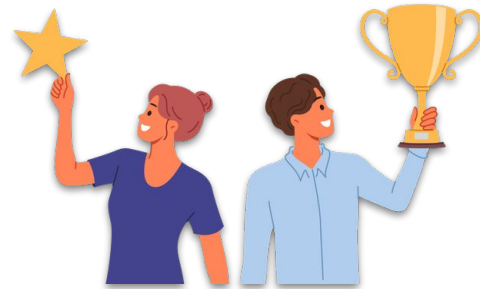
Inicio 17:20

Jueves dd/MM/YYYY



COMPETENCIA(S) QUE DESARROLLAREMOS

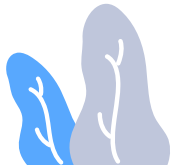
Comprende los conceptos, tipos y principios de las pruebas funcionales, valorando su rol dentro de una arquitectura de software para garantizar el cumplimiento de los requerimientos del sistema.



Puntualidad

Cumpliendo con responsabilidad y respeto por el tiempo: Honrando los compromisos adquiridos y valorando el tiempo propio y el de los demás, asegurando la presencia y entrega de resultados en los plazos establecidos. La puntualidad refleja organización, compromiso y profesionalismo en cada acción realizada.

Fortaleciendo la confianza y la eficiencia colectiva: Promoviendo un ambiente de respeto y orden, donde la disciplina en el cumplimiento de horarios contribuye a la coordinación de tareas y al logro de objetivos comunes. La puntualidad fomenta la confianza entre los integrantes del equipo y potencia la efectividad en los procesos compartidos.

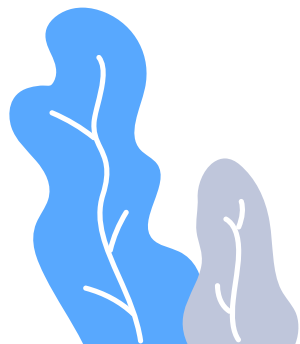


Agenda

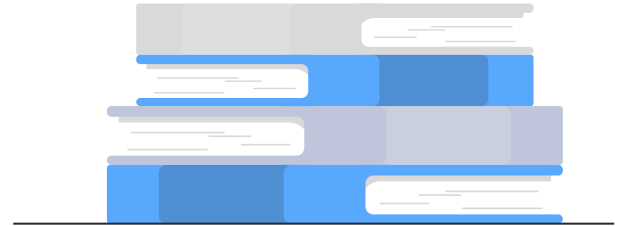
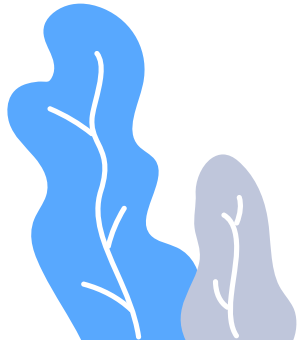
- 01 Enunciado Fase 3
- 02 Asistencia
- 03 Pruebas Funcionales
- 04 Dudas o comentarios
- 05 Foro de la semana



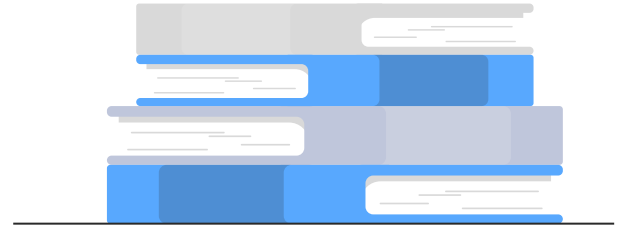
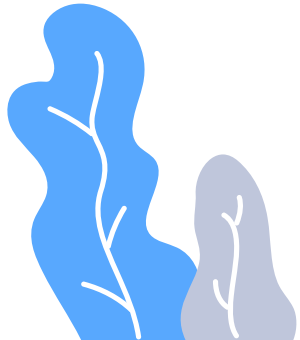
Enunciado Fase 3



Asistencia

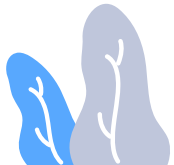


Pruebas de software



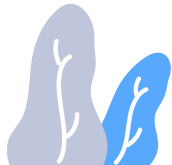
Introducción a las pruebas de software

Las pruebas de software son un proceso fundamental para garantizar la calidad y el correcto funcionamiento de los sistemas informáticos. Permiten identificar y corregir errores antes de que el software se ponga en producción, lo que ahorra tiempo y recursos.



¿Por qué son importantes las pruebas de software?

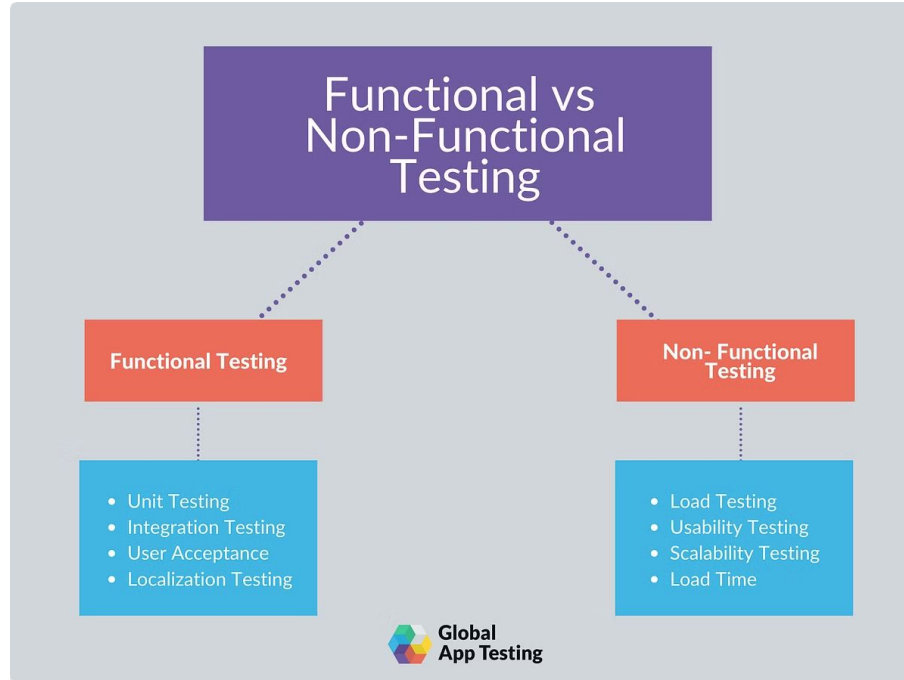
Las pruebas de software son fundamentales en el desarrollo de sistemas, ya que los defectos pueden afectar la reputación de una empresa, generar pérdidas económicas y provocar fallos críticos en sistemas interconectados. Un ejemplo de esto ocurrió en 2024, cuando un error en una actualización de software afectó a Delta Air Lines, causando cancelaciones masivas y pérdidas millonarias. La implementación de pruebas permite identificar problemas desde etapas tempranas, como errores de diseño, fallos funcionales, vulnerabilidades de seguridad y limitaciones de escalabilidad. De esta manera, se mejora la calidad del software, se reducen costos y se incrementa la satisfacción del usuario.



Tipos de pruebas de software

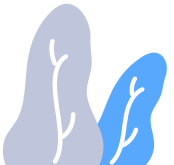
Pruebas Funcionales

Validan que el software cumpla con los requisitos especificados.



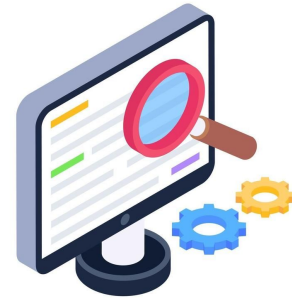
Pruebas No Funcionales

Evalúan aspectos como rendimiento, seguridad, usabilidad y compatibilidad.

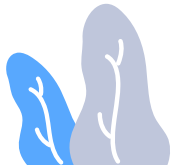


Pruebas Funcionales

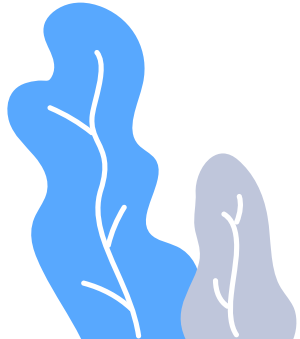
Las pruebas funcionales son un tipo de pruebas de software que verifican que el sistema cumpla con los requisitos funcionales establecidos, evaluando que cada funcionalidad opere correctamente según lo esperado, sin considerar su implementación interna.



Functional Testing



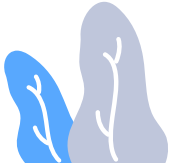
Tipos de pruebas funcionales



Pruebas unitarias

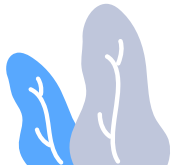
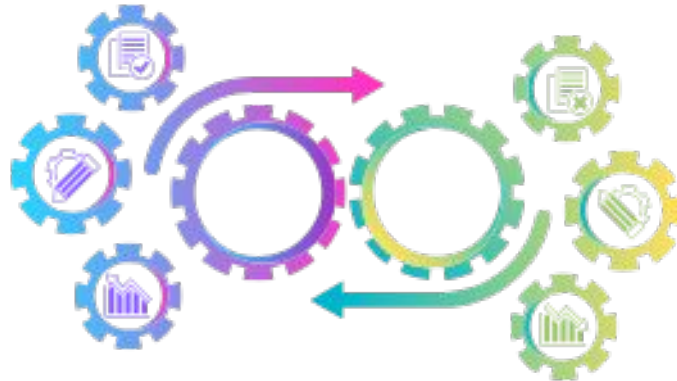
Las pruebas unitarias son la base de las pruebas funcionales. Se centran en verificar la menor parte testeable del software, como funciones y métodos individuales, para asegurarse de que se comporten como se espera. Es fundamental para detectar errores a nivel de componente antes de que el código avance a etapas de integración más complejas.

Se recomienda implementar pruebas unitarias durante las primeras etapas del desarrollo para garantizar una base de código sólida y confiable.



Pruebas de integración

Este tipo de pruebas evalúa la forma en que interactúan y operan varios módulos de aplicaciones de software de forma cohesiva. El sistema se divide en componentes conocidos como módulos o unidades. Cada módulo es responsable de una tarea específica. El verdadero desafío llega cuando combinamos estos componentes para desarrollar todo el sistema de software.



VENTAJAS

1

Resolución temprana
de defectos

2

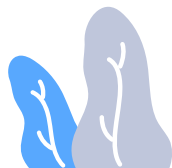
Identificar problemas
de integración
entre componentes

3

Mayor exhaustividad
que las pruebas
unitarias

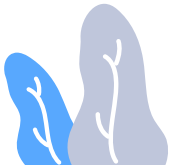
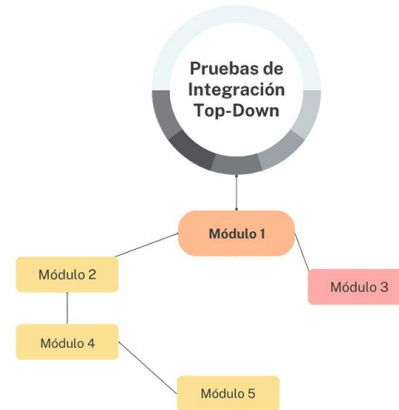
4

Mayor cobertura
de las pruebas



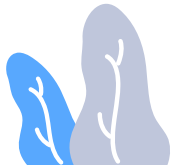
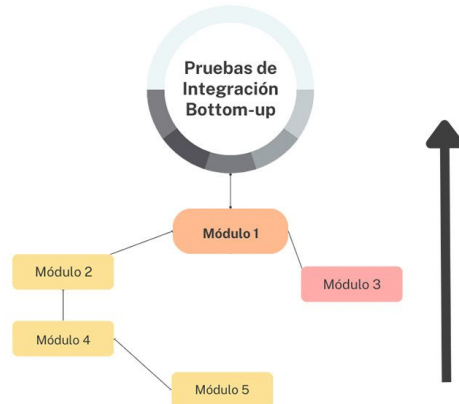
Integración Top-Down

Las pruebas top-down emplean un enfoque sistemático para probar los módulos de software desde el nivel superior hacia abajo a través de la jerarquía del sistema. Las pruebas comienzan con el módulo principal del software y continúan con los submódulos de la aplicación.



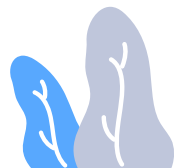
Integración Bottom-Up

Cuando se realizan pruebas bottom-up, primero se prueban los módulos de nivel inferior. Se pasa gradualmente a los módulos de nivel superior y así sucesivamente, hasta que todas las facetas del software se han probado a fondo. Esta estrategia se denomina razonamiento inductivo. Resulta beneficiosa cuando se incorporan al producto final componentes ya existentes.



Pruebas Unitarias vs. Pruebas de Integración

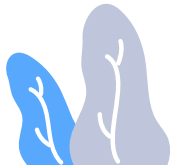
Aspecto	Pruebas Unitarias	Pruebas de integración
Alcance	Se concentran en unidades o módulos particulares.	Evalúan la interacción de módulos integrados.
Objetivo	Comprobar que cada elemento funciona de forma independiente.	Comprobar que todas las piezas conectadas entre sí funcionan correctamente.
Interoperabilidad	Las dependencias externas se mantienen separadas de las pruebas.	Se requiere la integración con módulos del mundo real.
Velocidad	Puesto que se concentra en unidades pequeñas, la ejecución es más rápida.	El tiempo de ejecución es un poco mayor debido a las pruebas de los diversos módulos.
Cobertura	Ofrece cobertura extensiva de código al inspeccionar módulos de forma exhaustiva.	Garantiza que los módulos funcionen correctamente como elemento del sistema genera.
Flujo de Trabajo de las Pruebas	Normalmente, los desarrolladores realizan pruebas unitarias antes que las pruebas de integración.	Estas suelen llevarse a cabo durante la fase de integración del software.



Pruebas de aceptación

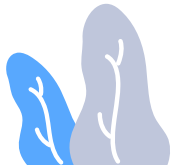
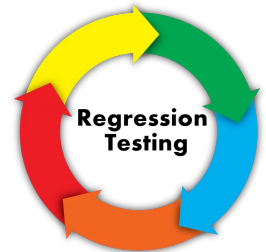
Las pruebas de aceptación son pruebas formales que verifican si un sistema satisface los requisitos empresariales. Requieren que se esté ejecutando toda la aplicación durante las pruebas y se centran en replicar las conductas de los usuarios. Sin embargo, también pueden ir más allá y medir el rendimiento del sistema y rechazar cambios si no se han cumplido determinados objetivos.

Caso de Prueba Aceptación	
Código: HU9_P10	Historia de Usuario: HU_9
Nombre: Listar y eliminar las notificaciones.	
Descripción: prueba para la funcionalidad de Listar las notificaciones.	
Condiciones de Ejecución: El usuario tiene que estar autenticado.	
Entrada/Pasos de Ejecución: una vez autenticado el usuario en el sistema, se debe acceder al submenú "Listar" del menú izquierdo "Notificaciones".	
Resultado Esperado: se creará una vista con el listado de todas las notificaciones que el usuario tiene registrada. Cada notificación dará la opción de ser eliminada. Si se da <i>click</i> en el botón eliminar, aparecerá un mensaje para confirmar la acción.	
Evaluación de la Prueba: prueba satisfactoria.	



Pruebas de regresión

Las pruebas de regresión se enfocan en verificar que los cambios, como las correcciones de errores, las mejoras de las funciones y las nuevas funciones añadidas no afecten negativamente las funciones existentes o la funcionalidad. Normalmente se realizan después de un cambio de código relevante o de haber desarrollado una nueva versión del software, y se emplean para asegurar que las funciones existentes sigan respondiendo según lo previsto. Las pruebas de regresión ayudan a garantizar que los nuevos cambios no generen un nuevo error ni provoquen fallos en las funciones existentes.



Ejemplo de prueba de regresión

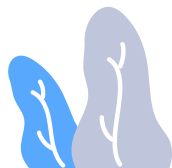
Cómo hacer Pruebas de Regresión

Veamos cómo realizar Pruebas de Regresión con un escenario del mundo real.

Supongamos que diriges tu propia empresa de desarrollo de software y te asignan un nuevo proyecto. A continuación, tu empresa comienza a implementar el producto basándose en los requerimientos que obtiene de los clientes. Al mismo tiempo, el equipo de pruebas comenzará a crear casos de prueba de acuerdo con los requerimientos del proyecto. Tras analizar los requerimientos, tu equipo de pruebas puede proponer 1000 casos de prueba.

Una vez que la implementación del producto ha finalizado, probarás el producto utilizando esos 1000 casos de prueba y entregarás el producto al cliente si los resultados de las pruebas son satisfactorios. En este punto, es probable que el cliente solicite añadir una nueva función. Ahora, tu equipo de pruebas tiene que escribir casos de prueba para evaluar la nueva función. Digamos que tu equipo de pruebas propone 50 casos de prueba solo para evaluar las funcionalidades de la nueva característica.

Se podría pensar que probar solamente los nuevos 50 casos de prueba es suficiente antes de entregar el producto al cliente. Aquí es donde entran en juego las pruebas de regresión. Cuando tu equipo de desarrollo añada nuevas características al producto, es posible que esto también cause errores en las funciones existentes. Por lo tanto, no solo deberías ejecutar los últimos 50 casos de pruebas sino también los 1000 que creó para el producto original. Sin embargo, en caso de que no tengas el tiempo suficiente para ello, puedes seleccionar solamente los casos de prueba que evalúen las funciones que puedan verse afectadas por el desarrollo de la nueva característica. Este proceso en general se denomina "pruebas de regresión".



A man with a beard and short dark hair, wearing a white lab coat over a grey shirt and a brown tie, is smiling and holding a glass beaker. The beaker contains a vibrant, multi-colored liquid (blue, green, yellow, and red) and a wisp of red smoke is rising from it. To his right, a magnifying glass with a blue handle and frame is positioned over a red ladybug icon. The background is dark blue with faint, repeating lines of white and red code. A yellow banner is at the bottom right.

**¿POR QUÉ DEBES
PROBAR TU CÓDIGO?**

Ejemplo Práctico



Actividad en Clase

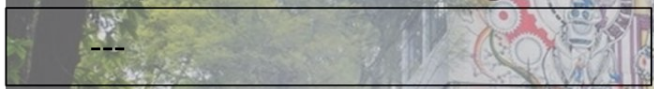
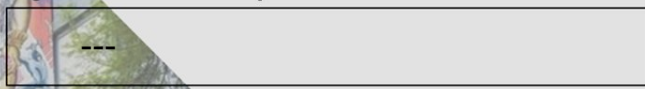
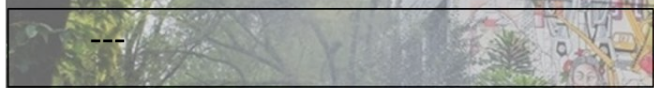
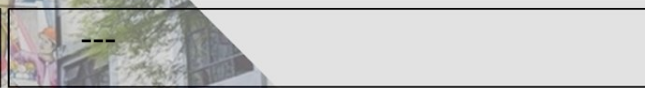
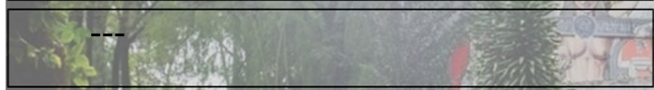
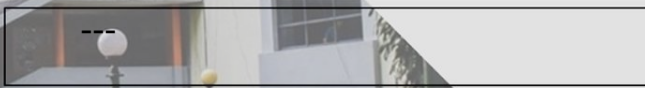
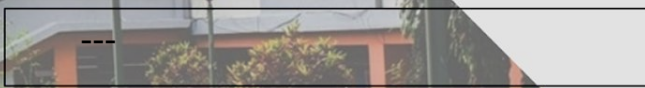
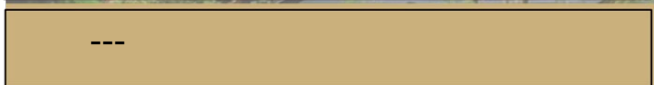
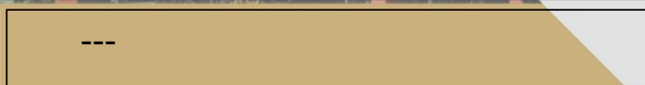




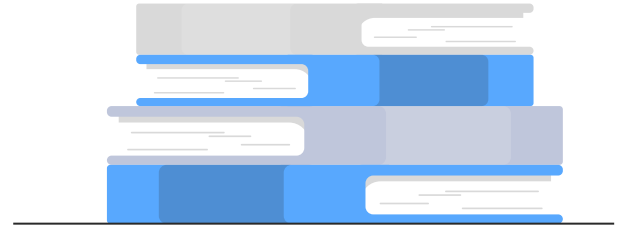
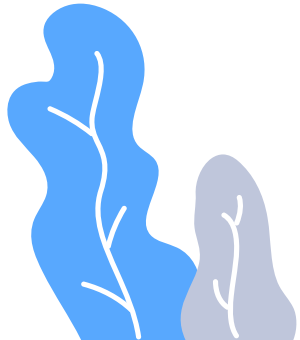
Nombre de la actividad:	---
Cantidad de participantes:	---
Doy fe que esta actividad está planificada en dtt (Sí/No):	---

Hora de inicio:	---
Hora de fin:	---
Duración (min):	---

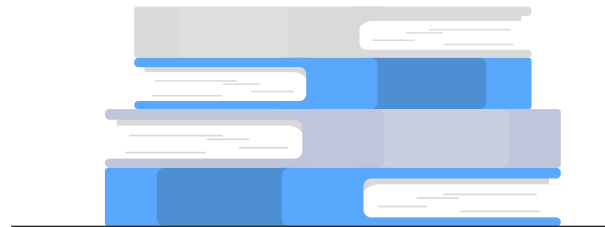
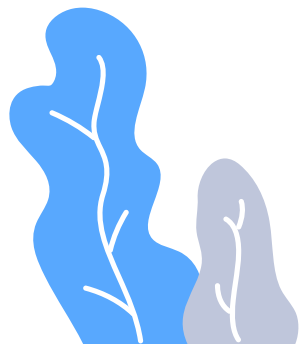
Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Dudas o comentarios



Foro de la semana



¡GRACIAS POR SU
ATENCIÓN!

